REAL WORLD RAG SYSTEM CAPSTONE

Domain-Specific RAG Evaluation and Optimization

with a Controlled KAG Enhancement Layer

Team members	Shiva | Sunil | Sourav | Vinay
Supervisor	Dr. Manish Shrivastava
Mentors	Gopichand and Lokesh
Submission date	30 May 2026
Scope	RAGBench + RGB mandatory work, with KAG as an innovation extension

Executive Summary

We propose to build an evaluation-driven RAG system across RAGBench domains, evaluate LLM generation behavior using RGB, and add a controlled Knowledge-Augmented Generation (KAG) enhancement for relationship-heavy domains such as finance and legal. The project prioritizes mandatory benchmark requirements first and introduces KAG as a measurable extension, not a replacement for RAGBench/RGB work.

Table of Contents

This table is static for proposal submission and aligned to the generated PDF version.

Executive Summary

This proposal defines a practical, evaluation-first plan for the Real World RAG System capstone. The team will build a modular RAG pipeline, evaluate it using RAGBench/TRACe metrics, evaluate generation behavior using RGB, and add a small KAG prototype to test whether structured knowledge improves results in relationship-heavy domains.

Positioning for full marks

We will complete the mandatory RAGBench and RGB requirements first. KAG is included as an additional controlled enhancement that can be compared against baseline and optimized RAG using the same evaluation discipline.

1. Proposal Snapshot

Area	Plan
Core objective	Build, evaluate, and improve a domain-specific RAG pipeline using RAGBench, then evaluate LLM behavior using RGB.
Mandatory benchmarks	RAGBench for end-to-end RAG evaluation; RGB for LLM generation capability evaluation.
Innovation layer	KAG - Knowledge-Augmented Generation using lightweight entity-relation graphs for selected domains.
Primary domains	All five RAGBench domains will be considered; KAG will start with finance/legal and extend if time permits.
Evaluation focus	TRACe metrics: context relevance, context utilization, answer completeness, adherence/faithfulness; RGB metrics: accuracy, rejection rate, error detection rate, error correction rate.
Deployment	Simple Streamlit or Gradio UI, with optional FastAPI backend for clean integration.

2. Problem Statement

Large Language Models can produce fluent answers but may hallucinate, rely on outdated knowledge, or fail when domain-specific evidence is required. Retrieval-Augmented Generation addresses this by retrieving relevant documents and grounding generation in external context. However, a RAG system is only reliable when the retriever returns the right evidence and the generator uses that evidence faithfully.

The challenge in this project is not just to build a RAG demo. The real objective is to evaluate and optimize the full RAG pipeline across multiple domains and task types. We will analyze how chunking, embeddings, vector stores, retrieval strategies, re-ranking, prompt design, and summarization affect measurable quality. We will then explore whether a KAG enhancement layer can improve reasoning over structured relationships, numbers, dates, entities, and multi-hop evidence.

3. Project Objectives

1. Understand the RAGBench and RGB benchmarks and summarize their relevance to real-world RAG evaluation.
2. Build a modular baseline RAG pipeline that can run across the five RAGBench domains.
3. Implement or reproduce the RAGBench/TRACe evaluation metrics and validate them using dataset-provided annotations and scores.

4. Experiment with improved chunking, embeddings, vector databases, retrieval methods, re-ranking, and context repacking.
5. Compare domain-wise performance and identify which configuration works best for each domain.
6. Evaluate LLM generation abilities on RGB: noise robustness, negative rejection, information integration, and counterfactual robustness.
7. Add a KAG prototype for selected domains to compare standard RAG versus graph-augmented retrieval.
8. Deliver a clean project report, GitHub repository, experiment matrix, and simple demo UI.

4. Dataset Description and Relevance

4.1 RAGBench

RAGBench is a large-scale benchmark designed to evaluate RAG systems using real-world RAG-style examples. It standardizes multiple component datasets into a common schema with question, documents, generated response, sentence-level support annotations, and evaluation scores. The benchmark covers five industry-oriented domains: biomedical research, general knowledge, legal, customer support, and finance.

Domain	Component datasets	Why it matters for our project
Biomedical research	PubMedQA, CovidQA	Requires careful handling of scientific context and evidence flow; good for testing semantic chunking and domain-specific retrieval.
General knowledge	HotpotQA, MS Marco, HAGRID, ExpertQA	Includes broad QA and multi-hop examples; useful for testing retrieval breadth and query reformulation.
Legal	CUAD	Long contract contexts and clause-level evidence make it useful for section-aware chunking, adherence, and KAG.
Customer support	DelucionQA, EManual, TechQA	Manuals and support documents mimic practical industry RAG scenarios; useful for issue-resolution retrieval.
Finance	FinQA, TAT-QA	Often requires numerical, temporal, and table/text reasoning; strong candidate for KAG.

The Hugging Face dataset viewer shows 12 subsets with approximate row counts, confirming that the dataset can be loaded and processed domain by domain.

Subset	Approx. rows	Domain
pubmedqa	24.5k	Biomedical research
covidqa	1.77k	Biomedical research
hotpotqa	2.7k	General knowledge
msmarco	2.69k	General knowledge
hagrid	4.53k	General knowledge
expertqa	2.03k	General knowledge
cuad	2.55k	Legal
delucionqa	1.83k	Customer support

Subset	Approx. rows	Domain
emanual	1.32k	Customer support
techqa	1.81k	Customer support
finqa	16.6k	Finance
tatqa	33.1k	Finance

4.2 Important RAGBench Fields

Field type	Examples	Use in our project
Input fields	id, question, documents, documents_sentences	Build indexes, retrieve context, and trace evidence back to sentence-level spans.
Generated output fields	response, response_sentences, generation_model_name	Compare generated responses and study model behavior.
Annotation fields	all_relevant_sentence_keys, all_utilized_sentence_keys, sentence_support_information	Reproduce TRACe metrics and validate evaluator logic.
Score fields	adherence_score, relevance_score, utilization_score, completeness_score	Use as reference scores and experiment comparison baselines.
Baseline evaluator fields	RAGAS, TruLens, GPT-3.5 scores where available	Optional comparison against existing automated evaluators.

4.3 RGB Benchmark

RGB evaluates whether LLMs behave reliably when external retrieved documents are noisy, incomplete, distributed across multiple sources, or factually wrong. This benchmark is separate from RAGBench: RAGBench evaluates the RAG pipeline, while RGB evaluates generation abilities under retrieval-augmented conditions.

RGB ability	Question being tested	Metric
Noise robustness	Can the LLM extract the correct answer when related noisy documents are present?	Accuracy
Negative rejection	Can the LLM refuse to answer when no document contains the answer?	Rejection rate
Information integration	Can the LLM combine evidence from multiple documents to answer a complex question?	Accuracy
Counterfactual robustness	Can the LLM detect false context and provide the correct answer when appropriate?	Error detection rate, error correction rate

5. Proposed Methodology

Our methodology follows the mentor recommendation: first complete the expected RAGBench pipeline and evaluation, then move to RGB, and finally use remaining time for KAG/bonus improvements. We will keep the system modular so each component can be replaced and compared.

Pipeline stage	Baseline choice	Planned improvements
Data loading and EDA	Hugging Face datasets + pandas profiling	Domain-wise sampling, schema normalization, metadata preservation.
Chunking	Sliding-window or recursive chunking as first working baseline	Semantic chunking, sentence-aware chunking, metadata-enriched chunks, small-to-big retrieval.
Metadata generation	Store domain, subset, doc ID, chunk ID, source sentence keys	Generate titles, keywords, summaries, and hypothetical questions for selected chunks.
Embeddings	Lightweight SentenceTransformer/MTEB-ranked model	Compare bge/e5/jina-style models depending on resources.
Vector DB	Chroma or FAISS	Compare Chroma metadata filtering; Qdrant/Milvus if setup time permits.
Retrieval	Dense top-k retrieval	BM25 sparse retrieval, hybrid retrieval, query rewriting, HyDE, query decomposition.
Re-ranking	Similarity score ordering	Cross-encoder or monoT5-style re-ranking if feasible.
Generation	Grounded prompt with instruction to avoid unsupported claims	Prompt variants for answer completeness and negative rejection.
Evaluation	TRACe metric reproduction and experiment matrix	Domain-wise analysis, RAG vs KAG comparison, RGB testbeds.

5.1 RAGBench Implementation Plan

1. Create a common dataset adapter that loads any RAGBench subset and standardizes question, documents, sentence keys, response, and metric fields.
2. Create a reusable chunking module with sliding window, sentence-aware, semantic, and metadata-enriched strategies.
3. Create vector indexes per domain/subset so experiments can be repeated and compared.
4. Run baseline dense retrieval and generation for representative samples from each domain.
5. Implement TRACe metric calculation from sentence-level annotations and score fields; validate against dataset scores.
6. Run controlled experiments where only one component changes at a time: chunking, embedding, vector DB, retrieval, re-ranker, or prompt.
7. Prepare a domain-wise result matrix and explain why certain combinations perform better or worse.

5.2 Evaluation Framework

For RAGBench we will use the TRACe framework: uTilization, Relevance, Adherence, and Completeness. We will compute or reproduce the metrics using sentence-level annotations wherever available.

Metric	Meaning	How we will use it
Context relevance	Fraction of retrieved context that is relevant to the query.	Diagnoses retriever quality and wasted context.
Context utilization	Fraction of retrieved context used by the generator.	Diagnoses whether the generator is actually using retrieved evidence.
Completeness	Fraction of relevant information that appears in the final answer.	Measures whether the answer covers all necessary evidence.

Metric	Meaning	How we will use it
Adherence / faithfulness	Whether every response claim is grounded in the retrieved context.	Detects hallucination and unsupported claims.

Additional retrieval diagnostics such as Recall@k, Precision@k, MRR@k, and latency will be captured when applicable. These will help explain why a specific RAGBench score changes after a component change.

6. KAG Enhancement Layer

KAG will be introduced as a controlled enhancement after the baseline RAG pipeline is working. The goal is to test whether structured knowledge improves retrieval and generation in domains where plain vector similarity may miss relationships, dates, numbers, obligations, or multi-hop dependencies.

Standard RAG	KAG-enhanced RAG
Retrieves similar text chunks from a vector DB.	Retrieves text chunks plus entity-relation triples from a lightweight knowledge graph.
Reasoning is mostly implicit inside the LLM prompt.	Important facts are explicitly represented as subject-relation-object triples.
Works well for broad document search.	Expected to help in legal, finance, biomedical, and multi-hop questions.
Can be confused by noisy but semantically similar context.	Can provide additional grounding through entities, dates, metrics, and relationships.

6.1 Practical KAG Scope

- We will not replace RAGBench/RGB work with KAG. KAG is an extension to strengthen the project.
- Initial KAG target: finance or legal. These domains are relationship-heavy and easier to demonstrate.
- If time permits, extend KAG to biomedical research or customer support.
- Use LLM-assisted extraction or rule-assisted extraction to generate triples from selected chunks.
- Store triples in a simple NetworkX graph or lightweight local graph structure. Neo4j will be considered only if setup time permits.
- At query time, extract query entities, retrieve relevant graph neighbors, and combine graph facts with vector-retrieved chunks.

KAG flow

Question -> query entity extraction -> vector retrieval + graph retrieval -> evidence fusion -> grounded generation -> TRACe evaluation

7. Experiment Matrix

Version	Configuration	Purpose
V1	Sliding-window chunks + baseline embedding + Chroma/FAISS + dense retrieval + grounded prompt	End-to-end baseline.
V2	V1 + semantic/sentence-aware chunking	Measure chunking impact.

Version	Configuration	Purpose
V3	Best chunking + improved embedding model	Measure embedding impact.
V4	Best chunking/embedding + BM25 and hybrid retrieval	Measure keyword + semantic retrieval impact.
V5	V4 + HyDE/query rewriting for selected questions	Improve retrieval for difficult queries.
V6	V4/V5 + re-ranking and context repacking	Improve ordering and relevance of evidence sent to the LLM.
V7	Best RAG + KAG graph facts for selected domain	Compare standard RAG vs KAG-enhanced RAG.
V8	RGB evaluation on selected open-source LLMs	Evaluate LLM capability behavior independently of RAGBench optimization.

8. Team Plan and Responsibilities

Team member	Primary responsibility	Secondary responsibility
Shiva	Project coordination, proposal/report quality, KAG design, final integration.	Architecture review, presentation storyline, mentor communication.
Sunil	Dataset loading, EDA, schema normalization, metadata strategy.	Domain-wise analysis and experiment documentation.
Sourav	RAG pipeline implementation: chunking, embeddings, vector DB, retrieval.	Hybrid retrieval and re-ranking experiments.
Vinay	Evaluation implementation: TRACe metrics, RGB metrics, experiment tracking.	Deployment UI, result tables, GitHub hygiene.

8.1 Working Model

- Maintain a GitHub repository with modular folders for loaders, chunkers, embedders, retrievers, re-rankers, generators, evaluators, KAG, UI, and experiments.
- Use local VS Code development with a shared virtual environment; use Colab/Kaggle only when GPU resources are needed.
- Log every experiment with configuration, domain, sample size, runtime, metrics, and observations.
- Review progress weekly and freeze working versions before mentor reviews.

Suggested repository structure:

```
rag-kag-capstone/
  data_loaders/     chunkers/     embedders/     vectorstores/
  retrievers/       rerankers/    generators/    evaluators/
  kag/              experiments/  ui/            reports/
```

9. Expected Timeline

Week	Focus	Expected output
Week 1	Finalize proposal, read papers, load Hugging Face dataset, complete EDA.	Dataset understanding note, schema map, initial GitHub repo.

Week	Focus	Expected output
Week 2	Build baseline RAG pipeline for one or two domains.	Working baseline with chunking, embedding, vector DB, retrieval, generation.
Week 3	Implement/reproduce TRACe evaluation and validate against RAGBench fields.	Evaluator functions, reference score checks, baseline metrics.
Week 4	Run chunking and embedding experiments across domains.	First experiment matrix and domain-wise observations.
Week 5	Add hybrid retrieval, HyDE/query rewriting, re-ranking, and context repacking.	Improved RAG results and comparison against baseline.
Week 6	Implement KAG prototype for finance/legal and compare with standard RAG.	RAG vs KAG comparison with qualitative and metric-based analysis.
Week 7	Run RGB evaluation for selected LLMs and analyze the four abilities.	RGB results: noise robustness, negative rejection, information integration, counterfactual robustness.
Week 8	Finalize UI, report, presentation, GitHub cleanup, and demo.	Final report, presentation, repository, demo application.

10. Deployment Plan

Deployment will be simple because the main project marks are expected to come from benchmark understanding, implementation quality, evaluation, and analysis. We will create a lightweight UI that demonstrates the best RAG configuration and, if completed, the KAG-enhanced mode.

- Preferred UI: Streamlit or Gradio.
- Optional backend: FastAPI if we need a clean service boundary.
- Demo features: select domain, enter question, choose RAG or KAG mode, display retrieved chunks, display graph facts when KAG is enabled, show generated answer, and show available evaluation outputs.
- Deployment target: local demo first; cloud deployment only if time permits.

11. Risks and Mitigation

Risk	Mitigation
Dataset size and compute cost	Use domain-wise sampling first; scale experiments only after pipeline correctness is confirmed.
Evaluation complexity	Start evaluator work early; validate using provided RAGBench annotations and scores before evaluating new outputs.
Too many combinations	Use a controlled experiment matrix and change one major component at a time.
KAG scope may grow too large	Limit KAG to one or two selected domains and use simple triples before graph database tooling.
Open-source LLM limitations	Use smaller models where possible; keep prompt and evaluator methodology documented.
Timeline pressure	Prioritize RAGBench baseline + evaluation first, RGB second, KAG/bonus third.

12. Success Criteria and Final Deliverables

- A working modular RAG pipeline across RAGBench domains.
- RAGBench/TRACe evaluation implementation or reproduction with clear validation notes.
- Domain-wise experiment matrix showing which configuration works better and why.
- RGB evaluation of selected LLMs across the four required abilities.
- KAG-enhanced RAG comparison for at least one selected domain, if time permits.
- Simple UI demonstrating retrieval, generation, and evidence display.
- Final technical report, presentation, and GitHub repository.

13. Glossary

Term	Meaning in this project
RAG	Retrieval-Augmented Generation: retrieving external context and using it to ground LLM responses.
KAG	Knowledge-Augmented Generation: adding structured entity-relation knowledge, such as graph triples, alongside vector retrieval.
LLM	Large Language Model used for answer generation, query rewriting, summarization, or entity extraction.
RAGBench	Benchmark dataset for end-to-end RAG evaluation across biomedical, general knowledge, legal, customer support, and finance domains.
RGB	Benchmark for evaluating LLM behavior under retrieval conditions: noise robustness, negative rejection, information integration, and counterfactual robustness.
TRACe	RAGBench evaluation framework covering uTilization, Relevance, Adherence, and Completeness.
Context relevance	How much of the retrieved context is actually relevant to the question.
Context utilization	How much of the retrieved context is used by the generator in the final answer.
Completeness	How well the final answer includes all relevant information available in the context.
Adherence / faithfulness	Whether all answer claims are grounded in the provided context.
Hallucination	Unsupported or factually incorrect content generated by the LLM.
Chunking	Splitting documents into retrievable pieces such as sentence-aware, sliding-window, or semantic chunks.
Embeddings	Vector representations of text used for semantic similarity search.
Vector database	Storage and search layer for embeddings, such as Chroma, FAISS, Qdrant, Milvus, or Weaviate.
BM25	Sparse keyword-based retrieval method useful for exact term matching.
Dense retrieval	Retrieval using embedding similarity.

Term	Meaning in this project
Hybrid retrieval	Combining sparse retrieval such as BM25 with dense vector retrieval.
HyDE	Hypothetical Document Embedding: generate a pseudo-answer/document first, then use it for retrieval.
Re-ranking	Reordering retrieved chunks using a second-stage relevance model or heuristic.
Knowledge graph	Structured representation of entities and relationships, often as subject-relation-object triples.
Entity-relation triple	A simple graph fact such as Company - reported revenue - Amount.
Recall@k / Precision@k / MRR@k	Retrieval diagnostics used to analyze whether relevant evidence appears in the top-k retrieved results and how early it appears.

14. Final Positioning

Our project will first satisfy the required capstone expectations: RAGBench, RGB, evaluation metrics, domain-wise experiments, deployment, report, and presentation. The KAG layer is included as an innovation component that gives the project a stronger research and practical angle. By comparing baseline RAG, optimized RAG, and KAG-enhanced RAG, we will produce not only a working application but also a clear technical analysis of what improves RAG quality in real-world domains.

References and Source Material

- RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems, arXiv:2407.11005v2.
- Benchmarking Large Language Models in Retrieval-Augmented Generation, arXiv:2309.01431v2.
- RAGBench dataset on Hugging Face: galileo-ai/ragbench.
- Real World RAG System project document and mentor slides.
- Searching for Best Practices in Retrieval-Augmented Generation, ACL Anthology / EMNLP 2024 reference shared in project material.